SPRINTS | AI & Machine Learning Capstone

Learner Assessment & Resource Management System (LEARMS)

System Architecture & Implementation Technical Report

Author / Consultant:	Mohammed Wael (Junior Solutions Consultant)
Primary Stakeholder:	Fatima Rashid, Director of Technical Operations
Target Organization:	Sprints EdTech (Cairo, Egypt / MENA Region)
Engagement Duration:	Two-Week Capstone Acceleration Program
Technical Stack:	Core Python 3.10+, SQLite3, Linux CLI, WSL2, Git
Release Version:	v1.0.0 Production Baseline
Date of Delivery:	November 2026

Mission Mandate: Architect and deploy a secure, standardized Python-driven backend application that eliminates data ingestion bottlenecks, replaces unversioned flat text files with normalized SQLite persistence, enforces fault-tolerant ingestion with complete error isolation, and hardens the hosting Linux environment with least-privilege group permissions.

1. Executive Summary

Sprints is a premier educational technology institution dedicated to bridging the critical gap between talent learning and high-impact employment across the Middle East and North Africa (MENA). As cohort sizes have scaled exponentially across specialized tracks (Data Science, AI & Machine Learning, Full Stack Web Development, Cloud Computing, and Cybersecurity), managing learner assessment metrics became an acute operational bottleneck. Learner scores were historically exported into fragmented flat text files, requiring systems administrators to manually ingest data, execute disjointed scripts, and modify permissions ad-hoc. This legacy model presented severe vulnerabilities: risk of data corruption, absence of centralized version control, zero auditability, and security exposures.

The **Learner Assessment & Resource Management System (LEARMS)** resolves these challenges by delivering an automated, object-oriented backend pipeline coupled with an ACID-compliant SQLite3 relational database (data/sprints_learners.db). Built exclusively with core Python standard libraries—honoring strict client constraints against heavy third-party dependencies—LEARMS provides a fault-tolerant ingestion pipeline, automated periodic quality audit sampling (% 10 modulus check), identity-preserving conflict resolution (ON CONFLICT DO UPDATE), robust parameterized query SQL injection defense, and hardened Linux primary group security administration (sprints_ops, dynamically assigned GID: 1003 in WSL2 reference).

2. Software Configuration Management: Git Strategy

Version control was established at the inception of Sprint 1 using Git to provide transparent traceability across all milestones. A modular repository structure enforces strict separation between code logic, data persistence, audit logs, system scripts, and verification tests.

2.1 Structured Commit Workflow

The project adopted the Conventional Commits specification. Frequent, logical commits were made after each subsystem was implemented and verified:

Sprint / Phase	Commit Type	Component Scope & Operational Rationale
Sprint 1 (Day 1)	chore(init)	Project charter, directory skeleton, and enterprise .gitignore.
Sprint 1 (Day 3)	docs(arch)	System architecture report, database schema, and ER diagrams.
Sprint 2 (Day 6)	feat(models)	OOP hierarchy: User base class, polymorphic Learner & Instructor models.
Sprint 2 (Day 7)	feat(db)	SQLite relational schema creation with normalized tables and indexes.
Sprint 2 (Day 8)	feat(ingest)	CSV parser, score float casting, and % 10 modulus audit mechanism.
Sprint 2 (Day 9)	feat(log)	Activity logging, exception handling, and transaction summaries.
Sprint 2 (Day 10)	test(core)	Automated test suite (models, ingestion, injection, fault tolerance).
Sprint 3 (Day 12)	ops(linux)	WSL Ubuntu hardening: apt-get sqlite3, chown sprints_ops, chmod 750.
Sprint 3 (Day 13)	docs(deck)	Executive capstone presentation slides and generated PDF deck.
Sprint 3 (Day 14)	release	Final video walkthrough, deployment sign-off, and GitHub remote sync.

3. Python Object-Oriented Programming (OOP) Models

Domain models in code/models.py strictly encapsulate Sprints business logic, stakeholder attributes, and academic grading policies while demonstrating OOP inheritance and polymorphism.

3.1 Base Class: User

The User class establishes the foundational interface for all actors in the platform. It encapsulates user_id, full_name, email (auto-derived if not provided), role, and created_at. It exposes polymorphic methods display_profile() and get_details(), alongside validate() to guarantee non-empty identity fields.

3.2 Subclasses & Polymorphism: Learner & Instructor

Learner Subclass: Specializes User with academic attributes: cohort_name, assessment_score (enforced as float), enrollment_status, and is_audited. It overrides display_profile() to render score formatting, letter grade evaluations (A+, A, B, C, D, F), and attention-grabbing audit status tags ([★ QUALITY AUDIT FLAGGED]).

Instructor Subclass: Extends User with specialization and assigned_cohorts. It overrides display_profile() to highlight teaching assignments and mentoring focus areas.

```
# Polymorphic invocation example in LEARMS ingestion engine for user_obj in [learner_instance, instructor_instance]: # Dynamic dispatch invokes subclass-specific display formatting logger.info(user_obj.display_profile()) payload = user_obj.get_details() # returns specialized dictionary
```

4. Database Schema Design (SQLite)

To support Sprints scaling cohort volume, raw records are normalized into data/sprints_learners.db. The schema strictly adheres to Third Normal Form (3NF), separating cohort metadata from individual learner records.

4.1 Relational Architecture & Entity Relationships

The relational design establishes a 1-to-Many (1:N) relationship between Cohorts and Learners. Referential integrity is guaranteed by enabling PRAGMA foreign_keys = ON; upon every connection, with cohort updates cascading (ON UPDATE CASCADE) and deletions restricted (ON DELETE RESTRICT) to prevent accidental record loss and preserve learner historical integrity. Indexes on learners.cohort_id and learners.is_audited provide sub-millisecond query response.

4.2 Data Dictionary

Table: Cohorts	Column	Type	Constraints	Description
	cohort_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Surrogate numeric primary key.
	cohort_name	TEXT	NOT NULL UNIQUE	Official descriptive cohort title.
	program_track	TEXT	NOT NULL	Curriculum domain (e.g. Data Science).
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record insertion timestamp.

Table: Learners	Column	Type	Constraints	Description
	learner_id	TEXT	PRIMARY KEY	Natural enterprise ID (e.g., SP-L-1001).
	full_name	TEXT	NOT NULL	Learner legal full name.
	cohort_id	INTEGER	NOT NULL, FK (ON DELETE RESTRICT ON UPDATE CASCADE)	Foreign key linking to Cohorts.cohort.
	assessment_score	REAL	NOT NULL, CHECK(0..100)	Exam grade cast strictly to float.

	enrollment_status	TEXT	NOT NULL, CHECK(Status)	Status ('Active', 'Graduated', 'Dropped').
	is_audited	INTEGER	NOT NULL DEFAULT 0	Audit flag set by modulus 10 check.
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record insertion timestamp.

5. Application Logic & Data Flow

The core data engine in code/main.py manages end-to-end ingestion, validation, and persistence.

5.1 Data Ingestion & Transformation

Raw cohort records in raw_cohort_data.csv are parsed line-by-line using Python's standard csv module. Headers are rigorously normalized (removing irregular spacing, casing, and dashes). Strings are stripped, empty rows skipped, and assessment scores explicitly cast using float(score_str). To guarantee idempotency and avoid destructive cascade replacements, records are upserted using INSERT INTO learners ... ON CONFLICT(learner_id) DO UPDATE SET, updating existing fields in place.

5.2 Automated Periodic Quality Audit Sampling (% 10)

To fulfill Sprints employer partner quality audit requirements through systematic sampling, every 10th processed valid learner (valid_count % 10 == 0) is flagged (is_audited = 1). In the 45-learner cohort dataset, exactly 4 learners (SP-L-1010, SP-L-1020, SP-L-1030, SP-L-1040) are flagged and registered into both the learners table and the dedicated system_audit_log table.

5.3 Fault-Tolerant Exception Handling & Logging

All file streaming and database transactions are protected by hierarchical try-except blocks. When encountering corrupt rows (e.g. non-numeric score 'INVALID_SCORE', negative values, missing cohort keys), the engine logs a descriptive warning to logs/application_activity.log and advances to the next row without halting execution. Furthermore, configure_logger() ensures mandatory logging initialization without silent failures.

6. Linux Configuration & Process Management

LEARMS is designed to operate seamlessly within Linux/WSL environments adhering to enterprise systems administration and security standards.

6.1 Package Management via apt-get

Native package managers ensure reliable dependency provisioning without root dependencies during runtime. The automated setup executes: sudo apt-get update && sudo apt-get install -y sqlite3 libsqlite3-dev, confirming SQLite version 3.46.1.

6.2 Security, Primary Group, and File Permissions

To safeguard learner assessment scores and prevent unauthorized tampering:
1. An administrative group sprints_ops (dynamically assigned GID 1003 in reference WSL2; GID numbers vary across Linux systems) was provisioned and assigned as the operator's **Primary Group** via usermod -g sprints_ops $USER (verified via id mw851).
2. Executable scripts and code files were locked to octal 750 (rwxr-x---).
3. The SQLite database and data files were set to octal 660 (rw-rw----), granting group members write and read permissions for everyday ingestion without root escalation.
4. Activity logs were assigned octal 660 (rw-rw----), and documentation set to 640 (rw-r-----).

6.3 Real-Time Process Monitoring & Telemetry

Active execution was monitored using ps -p $PID -o pid,ppid,user,group,%cpu,%mem,vsz,rss,stat,start,time,comm. Telemetry confirmed operator mw851 running under group sprints_ops, steady-state CPU utilization under 28%, and resident memory footprint under 21 MB RSS, guaranteeing the system will not starve host infrastructure even during heavy batch imports.

7. Conclusion & Deployment Sign-off

The LEARMS capstone implementation fully resolves the data ingestion bottleneck identified by Fatima Rashid. Through strict adherence to core Python OOP principles, normalized relational data modeling, fault-tolerant exception handling, and Linux least-privilege security hardening, LEARMS provides Sprints with an enterprise-ready foundation ready for immediate production deployment.

Deliverable Component	Status	Verification Artifact
Software Configuration Management	COMPLETED	Git commit history across all 3 sprints
OOP Architecture & Polymorphism	COMPLETED	code/models.py (User, Learner, Instructor)
Relational SQLite Normalization	COMPLETED	data/sprints_learners.db (3NF, FKs, Indexes)
Modulus Audit Sampling (% 10)	COMPLETED	Flagged 4 learners (SP-L-1010, 1020, 1030, 1040)
Fault-Tolerant Ingestion	COMPLETED	corrupted_sample_data.csv edge tests passed
Linux Package & Permission Hardening	COMPLETED	logs/linux_operations.log (apt, chmod, chown, ps)
Executive Presentation Deck	COMPLETED	docs/Capstone_Presentation.pdf
Recorded Walkthrough Video	COMPLETED	Demo_Walkthrough.mp4 & DEMO_SCRIPT.md
Automated Test Suite (23 Tests)	COMPLETED	23/23 unit & integration tests passing in < 0.40s